

THE COMPLETE

GEN AI ENGINEER ROADMAP

01

From Python & ML fundamentals to LLMs, RAG, Agents, MCP & production LLMOps — a single, structured, beginner-to-advanced path to crack your next Gen AI interview.

42 SLIDES

9 MODULES

100+ INTERVIEW Q&A

6 PROJECT IDEAS



Ritesh Rai

GEN AI ENGINEER

I build and ship Generative AI systems — from LLM-powered products to retrieval and agentic pipelines — and write about Gen AI engineering for the community on LinkedIn.

This guide is the structured, end-to-end path I wish existed when I started: every topic a Gen AI interview can touch, broken into beginner-friendly modules, with the actual questions you'll be asked and how to answer them.

[LINKEDIN: GEN AI CONTENT](#)

[MENTORSHIP ON TOPMATE](#)

What You'll Learn

Nine modules, sequenced the way a real Gen AI Engineer learns the stack — from first principles to production.

01 Foundations

Python for AI, core math & statistics, SQL & data handling.

02 Machine Learning

Supervised/unsupervised learning, algorithms, evaluation, feature engineering.

03 Deep Learning

Neural networks, CNNs, RNNs/LSTMs, optimization & regularization.

04 NLP & Transformers

Text preprocessing, embeddings, the Transformer & attention mechanism.

05 Generative AI / LLMs

LLM fundamentals, prompt engineering, fine-tuning (LoRA/QLoRA/RLHF), evaluation.

06 Retrieval-Augmented Generation

RAG architecture, vector databases, chunking & retrieval strategies.

07 Agents & MCP

Agentic AI, LangChain/LlamaIndex/CrewAI, Model Context Protocol.

08 LLMOps & Deployment

Docker, Kubernetes, AWS/GCP/Azure, model serving & monitoring.

09 Capstone & Career

System design case study, responsible AI, projects, interview strategy.

How To Use This Guide

This isn't a list to skim once. Treat it like a workbook.

01 Go in order, once

The modules build on each other. Python and math show up again inside Transformers and fine-tuning math, so don't skip Module 01.

02 Don't just read — type the code

For every concept slide, open a notebook and implement the smallest possible version yourself. Reading $\neq$ retention.

03 Treat Q&A slides as flashcards

Cover the answer, say it out loud in your own words, then check. Repeat 2–3 days before any interview.

04 Build before you apply

Pick 2–3 projects from Module 09 and ship them publicly on GitHub. One real RAG app beats five completed courses on a resume.

05 Use this guide as a checklist

Before an interview, scan every module title. If any topic feels shaky, that's exactly where to spend your next study hour.

Python Essentials

The 20% of Python that shows up in 80% of Gen AI code — data pipelines, model code, and agent logic.

01 Core data structures

Lists, dicts, sets, tuples — and when to use each. Dicts power almost every config and API payload you'll write.

02 Functions & decorators

Args/kwargs, closures, and decorators like `@staticmethod` — used everywhere in frameworks like LangChain & FastAPI.

03 OOP fundamentals

Classes, inheritance, and composition — every LLM framework (LangChain, LlamaIndex) is built as composable classes.

04 Iterators & generators

yield and lazy evaluation — critical for streaming LLM tokens without loading everything into memory.

05 Async & concurrency

async/await, asyncio — needed to call multiple LLM/API endpoints in parallel without blocking.

06 Core libraries

NumPy for arrays, Pandas for tabular data, Requests for APIs — the daily-driver toolkit.

Python Essentials — Interview Q&A

Q What's the difference between a list and a tuple, and when would you use each?

A Lists are mutable and used for collections that change (e.g. accumulating tokens); tuples are immutable and used for fixed, hashable records like coordinates or dict keys.

Q Explain `*args` and `**kwargs` with a real use case.

A `*args` collects extra positional arguments into a tuple, `**kwargs` collects extra keyword arguments into a dict. Used heavily in wrapper functions — e.g. a retry decorator that forwards any arguments to the wrapped LLM call.

Q What is a generator and why would you use one when streaming LLM output?

A A generator yields values lazily, one at a time, instead of building a full list in memory. Streaming an LLM response token-by-token to the UI uses exactly this pattern.

Q What's the GIL, and does it matter for an AI application?

A The Global Interpreter Lock allows only one thread to execute Python bytecode at a time. It limits CPU-bound multithreading, but for I/O-bound work like calling LLM APIs, `asyncio` or threads still give real speedups.

Math & Statistics for AI

You don't need a PhD — you need fluency in the handful of ideas that explain why models behave the way they do.

01 Linear algebra

Vectors, matrices, dot products, matrix multiplication — the literal computation inside every neural network layer.

02 Probability basics

Conditional probability, Bayes' theorem — the foundation of how a language model predicts the next token.

03 Calculus essentials

Derivatives & gradients — the mechanism behind backpropagation and how models 'learn'.

04 Descriptive statistics

Mean, variance, standard deviation, distributions — used constantly in data analysis and evaluation.

05 Cosine similarity

Measures the angle between two vectors — the exact metric used to compare embeddings in semantic search and RAG.

Math & Statistics — Interview Q&A

Q Why does a neural network need gradients to learn?

A The gradient tells you the direction and rate at which the loss changes with respect to each weight. Subtracting a fraction of the gradient (gradient descent) nudges weights toward lower loss — that nudging, repeated millions of times, is 'learning'.

Q What is cosine similarity and why is it preferred over Euclidean distance for embeddings?

A Cosine similarity measures the angle between two vectors, ignoring magnitude. Embeddings encode meaning in direction, not length, so two semantically similar texts can have different magnitudes but a small angle — cosine similarity captures that correctly.

Q Explain overfitting in one sentence a 5-year-old could understand.

A It's like memorizing the answers to last year's exam instead of understanding the subject — you ace that exact test but fail a new one.

SQL & Data Handling

Most real-world Gen AI systems sit on top of structured data — you'll query it constantly.

— KEY CONCEPTS

► SELECT, JOIN, GROUP BY

Core SQL for pulling and aggregating training/evaluation data from relational sources.

► Indexes

Speed up retrieval on large tables — the same underlying idea vector indexes use for embeddings.

► Data cleaning with Pandas

Handling nulls, duplicates, and schema drift before any data reaches a model.

— INTERVIEW Q&A

Q: Why would a Gen AI engineer need SQL?

To pull training data, build evaluation sets, and query metadata stores or feature tables that feed RAG and fine-tuning pipelines.

Q: INNER JOIN vs LEFT JOIN — what's the difference?

INNER JOIN returns only matching rows in both tables; LEFT JOIN returns all rows from the left table, with NULLs where there's no match on the right.

ML Fundamentals

Every LLM is, underneath, a very large machine learning model — these fundamentals never stop being relevant.

01 Supervised vs unsupervised learning

Supervised learns from labeled examples (input → output); unsupervised finds structure in unlabeled data (e.g. clustering).

02 Train / validation / test split

Train to fit the model, validate to tune it, test to get an honest, final performance estimate.

03 Bias-variance tradeoff

High bias underfits (too simple), high variance overfits (too sensitive to training data) — the central tension in model design.

04 Loss functions

The number a model is trying to minimize — MSE for regression, cross-entropy for classification and language modeling.

05 Regularization

L1/L2 penalties and dropout that discourage a model from memorizing noise in the training data.

ML Fundamentals — Interview Q&A

Q What's the bias-variance tradeoff, with an example?

A Bias is error from overly simple assumptions (underfitting); variance is error from being too sensitive to training data (overfitting). A linear model on non-linear data has high bias; a deep decision tree with no pruning has high variance.

Q Why do we need a separate validation set if we already have a test set?

A The validation set is used to tune hyperparameters and make modeling decisions during development. If you tune on the test set, your final performance estimate becomes optimistic and unreliable — the test set must stay untouched until the very end.

Q What's cross-entropy loss, and why is it used for language models?

A It measures the difference between the predicted probability distribution and the true distribution (the actual next token). Language modeling is fundamentally a classification problem over the vocabulary at each step, which is exactly what cross-entropy is built for.

Core ML Algorithms

Know these well enough to whiteboard them — they're still asked in almost every ML/Gen AI interview loop.

01 Linear & logistic regression

The simplest, most interpretable baselines for regression and classification respectively.

02 Decision trees & random forests

Tree-based models that split data on feature thresholds; forests average many trees to reduce variance.

03 Gradient boosting (XGBoost/LightGBM)

Builds trees sequentially, each correcting the errors of the last — dominant on tabular data.

04 Support Vector Machines

Finds the maximum-margin boundary between classes, with the kernel trick for non-linear separation.

05 K-Means & clustering

Unsupervised grouping of similar data points — used for customer segmentation, anomaly detection, and embedding analysis.

ML Algorithms — Interview Q&A

Q Random forest vs gradient boosting — how do they differ?

A Random forest builds many trees independently (in parallel) on bootstrapped data and averages them to reduce variance. Gradient boosting builds trees sequentially, where each new tree corrects the residual errors of the previous ones, reducing bias.

Q What is the kernel trick in SVMs?

A It implicitly maps data into a higher-dimensional space where a linear boundary can separate classes that aren't linearly separable in the original space — without ever computing that mapping explicitly.

Q How do you choose K in K-Means?

A Common approaches are the elbow method (plotting within-cluster variance vs K and looking for the bend) and the silhouette score, which measures how well-separated clusters are.

Feature Engineering & Evaluation

The unglamorous work that decides whether a model actually works in production.

— KEY CONCEPTS

► Feature scaling

Standardization/normalization so features with different ranges contribute fairly to distance-based models.

► Encoding categoricals

One-hot, label, and target encoding — turning categories into numbers a model can use.

► Evaluation metrics

Precision, recall, F1, ROC-AUC for classification; RMSE, MAE for regression — pick based on what an error actually costs.

— INTERVIEW Q&A

Q: Precision vs recall — when do you optimize for which?

Precision matters when false positives are costly (e.g. spam filters); recall matters when false negatives are costly (e.g. fraud or disease detection).

Q: Why can accuracy be a misleading metric?

On imbalanced data (e.g. 99% negative class), a model predicting 'negative' always gets 99% accuracy while being useless — precision/recall/F1 expose this.

Neural Network Basics

Neural networks are the engine inside every modern Gen AI model — this is the layer beneath the layer.

01 The perceptron

A single neuron: weighted sum of inputs + bias, passed through an activation function.

02 Activation functions

ReLU, Sigmoid, Tanh, GELU — introduce non-linearity so networks can learn complex patterns, not just straight lines.

03 Forward propagation

Data flows through layers, each transforming it, until a final prediction comes out.

04 Backpropagation

The chain rule applied layer by layer to compute how much each weight contributed to the error — then update it.

05 Weight initialization

Poor initialization causes vanishing/exploding gradients before training even starts; Xavier/He init fix this.

Neural Networks — Interview Q&A

Q Why do we need activation functions at all?

A Without them, stacking linear layers just collapses into one big linear function, no matter how deep the network is. Non-linear activations let the network approximate complex, non-linear patterns.

Q What's the vanishing gradient problem?

A In deep networks, gradients can shrink as they're propagated backward through many layers (especially with Sigmoid/Tanh), so early layers barely update. ReLU-family activations and residual connections largely solve this.

Q Explain backpropagation in two sentences.

A Backpropagation computes the gradient of the loss with respect to every weight by applying the chain rule layer by layer, from output back to input. Those gradients are then used by an optimizer (like Adam) to update the weights.

CNNs, RNNs & LSTMs

The architectures that came before Transformers — still essential context and still asked in interviews.

— KEY CONCEPTS

► CNNs

Convolutional filters that detect local patterns (edges, textures) — the backbone of computer vision and multimodal encoders.

► RNNs

Process sequences step-by-step, carrying a hidden state forward — the original approach to language modeling.

► LSTMs / GRUs

Gated RNN variants that mitigate vanishing gradients, letting the network 'remember' longer-range context.

— INTERVIEW Q&A

Q: Why did Transformers replace RNNs for language modeling?

RNNs process tokens sequentially, which is slow and struggles with long-range dependencies. Transformers process all tokens in parallel via self-attention, capturing long-range relationships directly and training far faster.

Q: What problem do LSTMs solve that plain RNNs don't?

Plain RNNs suffer from vanishing gradients over long sequences, losing earlier context. LSTMs use gates (forget/input/output) to control what information persists in the cell state over long sequences.

Optimization & Regularization

How training actually converges — and how you stop a model from just memorizing the training set.

— KEY CONCEPTS

► Gradient descent variants

SGD, Momentum, RMSProp, and Adam — Adam is the default for almost all modern deep learning and LLM training.

► Learning rate scheduling

Warmup + decay schedules stabilize early training and fine-tune convergence late in training — standard in LLM pretraining.

► Dropout & batch normalization

Dropout randomly disables neurons during training to prevent co-adaptation; batchnorm stabilizes the distribution of layer inputs.

— INTERVIEW Q&A

Q: Why is Adam the default optimizer for training LLMs?

Adam combines momentum (smoothing gradient direction) with per-parameter adaptive learning rates, which makes it robust and fast-converging across the huge, noisy gradients typical of large-scale deep learning.

Q: What does a learning rate warmup phase actually do?

It starts training with a very small learning rate and ramps it up, preventing large, destabilizing weight updates early on when gradients are still noisy and the model is far from converged.

NLP Fundamentals

Before LLMs, this is how machines turned text into numbers — and these ideas still live inside every tokenizer.

01 Tokenization

Splitting text into units (words, subwords, characters) a model can process — modern LLMs use subword tokenization (BPE).

02 Bag-of-Words & TF-IDF

Classic, frequency-based ways to represent text numerically — simple, fast, still useful for search baselines.

03 Word embeddings (Word2Vec, GloVe)

Dense vectors that capture word meaning, where similar words end up close together in vector space.

04 Stemming & lemmatization

Reducing words to a root form ('running' → 'run') to normalize text before classic NLP pipelines.

The Transformer Architecture

The single architecture behind GPT, BERT, Claude, Gemini, and essentially every modern LLM.

01 Self-attention

Lets every token in a sequence directly attend to every other token, weighing their relevance — this is what 'understands context'.

02 Multi-head attention

Runs several attention mechanisms in parallel, each learning to focus on different relationship types (syntax, coreference, etc.).

03 Positional encoding

Injects word-order information, since self-attention itself has no inherent sense of sequence order.

04 Encoder vs decoder

Encoders (BERT) build bidirectional context for understanding; decoders (GPT) generate text autoregressively, left to right.

05 Feed-forward & residual connections

Per-token transformation layers plus skip connections that keep gradients flowing through very deep stacks.

Transformers — Interview Q&A

Q Explain self-attention like you would to a beginner.

A For every word, the model asks: 'which other words in this sentence should I pay attention to, to understand my meaning here?' It computes a relevance score against every other word and blends their information accordingly — that's attention.

Q Why do Transformers need positional encoding?

A Self-attention treats input as an unordered set of tokens — it has no built-in notion of word order. Positional encodings add order information to each token's embedding so the model knows 'cat sat on mat' differs from 'mat sat on cat'.

Q What's the difference between encoder-only, decoder-only, and encoder-decoder models?

A Encoder-only (BERT) builds bidirectional context, best for understanding/classification tasks. Decoder-only (GPT family) generates text autoregressively, best for open-ended generation. Encoder-decoder (T5) reads input fully, then generates output — best for tasks like translation and summarization.

Q Why is attention computation quadratic in sequence length, and why does that matter?

A Every token attends to every other token, so cost grows as $O(n^2)$ with sequence length n . This is exactly why long-context LLMs need specialized attention (e.g. sparse, sliding-window, or FlashAttention) to stay efficient.

LLM Fundamentals

What a 'large language model' actually is, and how it goes from raw text to a helpful assistant.

01 Pretraining

The model learns to predict the next token over massive text corpora — this is where 'world knowledge' is acquired.

02 Instruction tuning

Fine-tuning the pretrained model on (instruction, response) pairs so it follows commands instead of just continuing text.

03 RLHF / alignment

Reinforcement Learning from Human Feedback shapes the model toward helpful, honest, harmless responses using human preference data.

04 Context window

The maximum number of tokens a model can 'see' at once — directly limits how much conversation/document fits in one call.

05 Temperature & sampling

Temperature, top-k, and top-p control how random vs deterministic the model's token choices are at generation time.

LLM Fundamentals — Interview Q&A

Q What's the difference between pretraining, fine-tuning, and prompting?

A Pretraining builds general language ability from massive raw text. Fine-tuning further trains the model's weights on a smaller, task-specific dataset. Prompting changes no weights at all — it just steers the frozen model's behavior through input text.

Q What does temperature = 0 actually do?

A It makes generation effectively deterministic by always picking the highest-probability next token (greedy decoding) — useful for tasks needing consistency, like classification or code generation.

Q Why do LLMs hallucinate?

A An LLM is trained to produce plausible, fluent continuations — not verified facts. When it lacks reliable knowledge on a topic, it still generates a fluent-sounding answer because fluency, not truth, is what the training objective directly optimizes.

Prompt Engineering

The fastest, cheapest lever for improving LLM output — no training required.

01 Zero-shot vs few-shot prompting

Zero-shot asks the model directly; few-shot includes 2–5 examples in the prompt to demonstrate the desired pattern.

02 Chain-of-Thought (CoT)

Asking the model to 'think step by step' before answering — significantly improves performance on reasoning tasks.

03 ReAct prompting

Interleaves Reasoning and Acting — the model thinks, takes a tool action, observes the result, and repeats. Foundation of agents.

04 System prompts

Persistent instructions that set the model's role, tone, and constraints for the whole conversation.

05 Structured output prompting

Forcing JSON/schema-constrained output so an application can reliably parse the model's response.

Prompt Engineering — Interview Q&A

Q What is Chain-of-Thought prompting and why does it help?

A It prompts the model to reason step-by-step before giving a final answer. This works because it gives the model more 'computation steps' to arrive at a correct answer, instead of jumping straight to a guess.

Q Few-shot vs fine-tuning — when would you choose one over the other?

A Few-shot is fast, free, and great when you have a handful of examples and need flexibility. Fine-tuning is worth it when you need consistent behavior at scale, want to reduce prompt length/cost, or need a pattern too complex to demonstrate in a few examples.

Q How would you stop an LLM from going off-topic in a customer support bot?

A A strong system prompt defining scope and refusal behavior, combined with structured output constraints and, if needed, a guardrail/classifier layer that checks the response before it's shown to the user.

Fine-tuning & PEFT

How to adapt a base model to your domain without retraining billions of parameters from scratch.

— KEY CONCEPTS

► Full fine-tuning

Updates every model weight — most powerful, but expensive and requires significant GPU memory.

► LoRA

Freezes the base model and trains small, low-rank 'adapter' matrices injected into each layer — a fraction of the parameters, similar results.

► QLoRA

LoRA combined with 4-bit quantization of the base model — lets you fine-tune large models on a single consumer GPU.

► RLHF / DPO

Aligns model behavior to human preferences; DPO (Direct Preference Optimization) achieves similar results to RLHF without a separate reward model.

— INTERVIEW Q&A

Q: Why is LoRA so much cheaper than full fine-tuning?

It freezes ~99% of the original weights and only trains small low-rank matrices added on top, drastically cutting trainable parameters, GPU memory, and storage per fine-tuned variant.

Q: What's the practical difference between RLHF and DPO?

RLHF trains a separate reward model on human preferences, then optimizes the LLM against it with reinforcement learning (complex, unstable). DPO directly optimizes the LLM on preference pairs using a simpler, more stable loss — no separate reward model or RL loop needed.

Evaluation, Hallucination & Guardrails

Shipping an LLM feature means proving it's reliable — not just that it sounds good in a demo.

— KEY CONCEPTS

► LLM-as-a-judge

Using a strong LLM to score another model's outputs against a rubric — scalable, but needs calibration against human judgment.

► Standard benchmarks

MMLU, HellaSwag, HumanEval, etc. — useful for comparing base models, less useful for your specific application.

► Guardrails

Input/output filters, regex or classifier-based moderation, and schema validation that catch unsafe or malformed responses before users see them.

► Groundedness checks

Verifying a RAG answer is actually supported by the retrieved context — the core defense against hallucination in RAG systems.

— INTERVIEW Q&A

Q: How do you evaluate an LLM feature before shipping it?

Build a labeled evaluation set representative of real user queries, define task-specific metrics (accuracy, groundedness, latency, cost), and run both automated (LLM-as-judge) and periodic human evaluation before and after every prompt/model change.

Q: What's the difference between a guardrail and a hallucination check?

A guardrail filters for unsafe, off-topic, or malformed content regardless of correctness. A hallucination/groundedness check specifically verifies factual claims in the output are supported by the provided source or context.

RAG Architecture

Retrieval-Augmented Generation — how you give an LLM accurate, up-to-date, private knowledge without retraining it.

01 Why RAG exists

LLMs have a fixed knowledge cutoff and no access to private data — RAG retrieves relevant documents at query time and feeds them into the prompt.

02 Indexing pipeline

Documents are split into chunks, embedded into vectors, and stored in a vector database ahead of time.

03 Retrieval pipeline

The user query is embedded, the most similar chunks are retrieved, and those chunks are inserted into the LLM's context window.

04 Generation step

The LLM generates an answer grounded in the retrieved context instead of relying purely on its parametric memory.

05 Re-ranking

A second, more precise model re-orders the initially retrieved chunks to push the most relevant ones to the top before generation.

RAG — Interview Q&A

Q Why use RAG instead of just fine-tuning the model on your documents?

A RAG keeps knowledge separate from the model and updates instantly when source documents change — no retraining needed. It's also more transparent, since you can cite exactly which chunk an answer came from, and cheaper to maintain than continuous fine-tuning.

Q What causes a RAG system to give a wrong answer, and how do you debug it?

A Usually one of three failure points: retrieval (wrong/irrelevant chunks returned), context (right chunks but model misreads them), or generation (model ignores context and hallucinates). Debug by inspecting retrieved chunks first — most RAG failures are retrieval failures, not generation failures.

Q What's re-ranking and why add it to a RAG pipeline?

A Initial vector retrieval is fast but approximate. A re-ranker (often a cross-encoder) scores the top-K candidates more precisely against the query, improving the relevance of what's actually sent to the LLM — directly improving answer quality.

Vector Databases & Chunking

The two decisions that quietly determine whether your RAG system is great or mediocre.

— KEY CONCEPTS

► Vector databases

Pinecone, Weaviate, Chroma, FAISS, Milvus — store embeddings and support fast approximate nearest-neighbor search at scale.

► Chunking strategy

Fixed-size, sentence-based, or semantic chunking — chunk size directly trades off context completeness vs retrieval precision.

► Chunk overlap

Slightly overlapping consecutive chunks prevents important context from being split awkwardly across a chunk boundary.

► Hybrid search

Combining dense (embedding) search with sparse (keyword/BM25) search — catches both semantic and exact-match queries.

— INTERVIEW Q&A

Q: How do you choose a chunk size for RAG?

Balance completeness against precision: too large and irrelevant text dilutes the embedding and wastes context window; too small and you lose surrounding context. 200–500 tokens with some overlap is a common, reasonable starting point to tune from.

Q: What is approximate nearest neighbor (ANN) search, and why not exact search?

ANN algorithms (like HNSW) trade a small amount of accuracy for massive speed gains, making sub-second search possible across millions of vectors — exact search doesn't scale at that volume.

AI Agents Fundamentals

Moving from 'LLM answers a question' to 'LLM plans and takes actions to complete a goal.'

01 What makes something an agent

An LLM that can reason, choose from a set of tools/actions, observe results, and decide its next step — not just generate one-shot text.

02 The ReAct loop

Reason → Act → Observe → repeat, until the goal is achieved or a stopping condition is hit.

03 Tool / function calling

Letting the model invoke external functions (search, calculator, database query, API call) with structured arguments.

04 Memory

Short-term (conversation context) vs long-term (vector store or database) memory that persists across sessions.

05 Multi-agent systems

Multiple specialized agents (planner, researcher, coder) collaborate, each handling part of a larger task.

Agent Frameworks

You rarely build agent infrastructure from scratch — these frameworks handle the orchestration plumbing.

— KEY CONCEPTS

► LangChain / LangGraph

The most widely adopted framework for chaining LLM calls, tools, and memory; LangGraph adds explicit, controllable state graphs for complex agent flows.

► LlamaIndex

Optimized primarily for data indexing and retrieval — often the engine behind the RAG layer inside an agent.

► CrewAI / AutoGen

Frameworks purpose-built for orchestrating multiple cooperating agents with defined roles.

— INTERVIEW Q&A

Q: When would you choose LangGraph over plain LangChain chains?

When the workflow needs branching, loops, or human-in-the-loop checkpoints — LangGraph models the agent as an explicit state graph, which is far easier to debug and control than implicit chain logic.

Q: What's the risk of giving an agent too many tools?

Tool selection accuracy drops as the tool list grows — the model has to choose correctly among more options, increasing the chance of picking the wrong tool or misusing it. Narrow, well-described toolsets perform more reliably.

MCP — Model Context Protocol

An open standard for connecting LLMs to external tools and data — quickly becoming the 'USB-C for AI agents.'

— KEY CONCEPTS

► The problem MCP solves

Before MCP, every app wrote custom, one-off integration code for each tool/data source it wanted an LLM to use — an N×M integration problem.

► MCP servers & clients

An MCP server exposes tools/data in a standard format; any MCP-compatible client (like Claude) can discover and call them without custom glue code.

► Why it matters for engineers

You build one MCP server for your internal tool/API once, and it becomes usable by any MCP-compatible AI application — not just the one you built it for.

— INTERVIEW Q&A

Q: In one sentence, what problem does MCP solve?

It standardizes how AI applications connect to external tools and data sources, replacing custom one-off integrations with a single, reusable protocol.

Q: How is MCP different from a regular function/tool call?

A tool call is app-specific code wiring one model to one tool. MCP defines a standard client-server protocol, so any MCP-compatible model can use any MCP server's tools without bespoke integration work.

Docker for AI Apps

Packaging your model/app so it runs identically on your laptop, a teammate's machine, and production.

— KEY CONCEPTS

► Images & containers

An image is a frozen, reproducible snapshot of your app + dependencies; a container is a running instance of that image.

► Dockerfile basics

Defines the base image (e.g. python:3.11-slim), installs dependencies, copies code, and sets the start command.

► GPU containers

Using nvidia-docker / CUDA base images so containers can access the GPU for model inference.

— INTERVIEW Q&A

Q: Why containerize an LLM application instead of just running it on a VM?

Containers guarantee the exact same dependency versions and environment everywhere — eliminating 'it works on my machine' issues — and make scaling, rollback, and CI/CD dramatically simpler.

Q: What's the difference between an image and a container?

An image is the static, versioned blueprint (code + dependencies + config); a container is a live, running process created from that image — you can run many containers from one image.

Kubernetes for AI Apps

Once you have more than one container, Kubernetes is how you keep them running reliably at scale.

— KEY CONCEPTS

► Pods, Deployments, Services

A Pod runs one or more containers; a Deployment manages Pod replicas and rollouts; a Service gives them a stable network address.

► Autoscaling

Horizontal Pod Autoscaler adjusts replica count automatically based on CPU/GPU load or request volume — critical for handling LLM traffic spikes.

► GPU scheduling

Kubernetes can schedule pods onto nodes with available GPUs, sharing expensive GPU resources efficiently across workloads.

— INTERVIEW Q&A

Q: Why would a Gen AI team need Kubernetes instead of just Docker?

Docker runs containers; Kubernetes orchestrates many containers across many machines — handling autoscaling, self-healing (restarting crashed pods), load balancing, and zero-downtime rollouts that production LLM traffic needs.

Q: How does Kubernetes help control GPU costs?

Through autoscaling (scale inference pods down when idle) and bin-packing multiple workloads onto shared GPU nodes, instead of leaving expensive GPUs reserved and idle.

Cloud Platforms for AI

Every major cloud now has a managed layer purpose-built for Gen AI workloads.

— KEY CONCEPTS

► AWS

Bedrock for managed foundation models, SageMaker for training/hosting custom models, S3 for data, EC2/ECS for compute.

► Google Cloud

Vertex AI for the full model lifecycle (training, tuning, deployment, evaluation), and access to Gemini models.

► Microsoft Azure

Azure OpenAI Service for hosted GPT models with enterprise compliance, plus Azure ML for the broader ML lifecycle.

— INTERVIEW Q&A

Q: Why would a company use a managed service like Bedrock or Vertex AI instead of self-hosting a model?

Managed services remove the burden of provisioning GPUs, scaling, patching, and security — you get an API endpoint with built-in reliability, compliance, and pay-per-use pricing, trading some control for speed and lower ops overhead.

Q: When would you self-host a model instead of using a managed API?

When you need data to never leave your infrastructure, require a fine-tuned open-weight model, need to control cost at very high volume, or need latency/customization a managed API can't offer.

Model Serving & Monitoring

Getting a model into an API endpoint is easy. Keeping it fast, cheap, and observable is the real engineering.

— KEY CONCEPTS

► Serving engines

vLLM and TGI (Text Generation Inference) optimize LLM serving with techniques like continuous batching and PagedAttention for high throughput.

► Latency vs throughput

Optimizing for one often costs the other — batching improves throughput but can add latency for individual requests.

► Observability

Logging prompts, responses, latency, token usage, and cost per request — essential for debugging and controlling spend.

► Caching

Caching repeated or semantically similar queries (semantic caching) to cut both latency and API cost.

— INTERVIEW Q&A

Q: What does vLLM do differently from naive model serving?

It uses PagedAttention to manage the KV cache far more efficiently and continuous batching to keep GPUs busy across many concurrent requests — delivering significantly higher throughput than naive serving loops.

Q: How would you control runaway LLM API costs in production?

Track token usage and cost per request/user, set rate limits and budgets per user/feature, cache repeated queries, and route simpler queries to smaller/cheaper models instead of always using the largest model.

System Design: Production RAG Chatbot

A walkthrough of how the pieces fit together — this exact structure is a common system design interview question.

01 Ingestion layer

Documents are uploaded, parsed, chunked, embedded, and written to a vector database, with metadata for filtering.

02 API & retrieval layer

User query comes in → query is embedded → top-K chunks retrieved → optionally re-ranked.

03 Generation layer

Retrieved chunks + conversation history + system prompt are assembled into context and sent to the LLM.

04 Guardrails & evaluation

Output is checked for groundedness/safety before being returned; logs feed an evaluation pipeline for continuous improvement.

05 Infra layer

Containerized services on Kubernetes, autoscaled, with caching, monitoring, and cost tracking wrapped around every call.

Responsible AI & Security

The questions interviewers ask to see if you've actually shipped AI to real users, not just a notebook.

— KEY CONCEPTS

► Prompt injection

Malicious input designed to override your system prompt or extract hidden instructions/data — a top security risk for any LLM app with untrusted input.

► Data privacy

Avoiding sending PII to third-party model APIs without consent, and respecting data residency/compliance requirements.

► Bias & fairness

Models can reproduce biases present in training data — evaluation sets should explicitly test for this, not just overall accuracy.

— INTERVIEW Q&A

Q: What's prompt injection, and how do you defend against it?

It's when user input tries to override your system instructions (e.g. 'ignore previous instructions and...'). Defenses include strict system/user role separation, input sanitization, output validation, and never granting the model more tool permissions than the specific task needs.

Q: How do you handle PII when building a RAG system over customer data?

Redact or mask PII before embedding/indexing where possible, restrict retrieval with access controls per user/tenant, and ensure your model provider's data retention policy matches your compliance requirements.

Project Ideas — Build Your Portfolio

Six projects, beginner to advanced. Build at least 3 of these end-to-end before interviews — recruiters trust GitHub more than résumé bullets.

BEGINNER**Document Q&A Bot**

A RAG app that answers questions over your own PDFs using a vector DB and an LLM API.

Python • LangChain • Chroma • OpenAI/Claude API

BEGINNER**Prompt-Engineered Resume Reviewer**

An app that critiques resumes against a job description using structured, few-shot prompting.

Python • Streamlit • LLM API

INTERMEDIATE**Multi-Source RAG with Re-ranking**

RAG over multiple data types (PDFs + web + DB) with hybrid search and a re-ranking step.

LlamaIndex • Pinecone • Cross-Encoder

INTERMEDIATE**Fine-tuned Domain Chatbot**

Fine-tune an open-weight model with LoRA/QLoRA on a niche dataset (e.g. legal, medical, support tickets).

PEFT • Hugging Face • QLoRA

ADVANCED**Autonomous Research Agent**

A multi-step agent that plans, searches the web, calls tools, and writes a cited report.

LangGraph • Function Calling • Web Search Tool

ADVANCED**Production LLM Service on Kubernetes**

Containerize a fine-tuned model with vLLM, deploy on K8s with autoscaling, caching, and full observability.

Docker • Kubernetes • vLLM • Prometheus

Interview Prep Strategy

How to actually convert this roadmap into offers.

01 Map topics to the job description

Re-read the JD and circle every term that maps to a module here — that's your priority study order, not module order.

02 Prepare 2 project stories in depth

For your strongest 1–2 projects, be ready to explain every design decision — why this chunk size, why this vector DB, what you'd change.

03 Practice explaining trade-offs out loud

Interviewers care less about the 'right' answer and more about whether you can reason about trade-offs — practice saying it, not just knowing it.

04 Do timed mock interviews

Knowing an answer and producing it clearly under time pressure are different skills — rehearse the second one specifically.

05 Keep a running 'gaps' list

Every time you blank on a question while studying, write it down. Review that list the night before every interview.

Knowing the roadmap is step one. Knowing how to crack YOUR interview is step two.

I work 1:1 with beginners and career switchers on resume reviews, mock interviews, and personalised Gen AI learning plans — based on where you are and where you're applying.

Book a 1:1 session with me on Topmate

<https://topmate.io/dashboard/services/video>

Found this useful? Follow **Ritesh Rai** on LinkedIn for more Gen AI breakdowns, every week.